POMPS Presentation: 1.2.55

Subhodeep Chakraborty
ee25btech11055,
IIT Hyderabad.

March 25, 2026

1. Problem
2. Solution
3. Python Codes
4. Plot

Problem Statement

Question:

The minimum number of terms required in e^x series expansion to evaluate at $x = 1$ correct up to 3 decimal places is:

1. 8
2. 7
3. 6
4. 5

Matrix Operator Representation

Let $\mathbf{S}$ be the infinite-dimensional downshift matrix, representing a discrete unit delay operator (z^{-1}).

When multiplied by a vector, it shifts all elements down by one position:

$$\mathbf{S} = \begin{bmatrix} 0 & 0 & 0 & \dots \\ 1 & 0 & 0 & \dots \\ 0 & 1 & 0 & \dots \\ \vdots & \vdots & \ddots & \ddots \end{bmatrix}, \quad \mathbf{S} \begin{bmatrix} a \\ b \\ c \\ \vdots \end{bmatrix} = \begin{bmatrix} 0 \\ a \\ b \\ \vdots \end{bmatrix} \tag{3.1}$$

Ideal System Response

The ideal infinite impulse response of this system is generated by applying the matrix exponential to an impulse vector $\delta = [1, 0, 0, \dots]^T$.

Expanding the Taylor series of $e^{\mathbf{S}}$ directly generates the Maclaurin coefficients:

$$\mathbf{h}_{\text{ideal}} = e^{\mathbf{S}}\delta = \left(\mathbf{I} + \mathbf{S} + \frac{1}{2!}\mathbf{S}^2 + \frac{1}{3!}\mathbf{S}^3 + \dots\right)\begin{bmatrix}1\\0\\0\\\vdots\end{bmatrix} = \begin{bmatrix}1\\1\\1/2!\\1/3!\\\vdots\end{bmatrix} \tag{3.2}$$

Filter Truncation and Error Matrix

Truncating this infinite system to a finite filter length N yields the error matrix, $\mathbf{E}_N$:

$$\mathbf{E}_N = \sum_{k=N}^{\infty} \frac{1}{k!} \mathbf{S}^k \tag{3.3}$$

To measure the maximum size of this error, we apply the induced l_1-norm ($\|\cdot\|_1$), defined as the maximum absolute column sum of a matrix. We use this instead of Frobenius as we have an infinite matrix and this norm looks for worst-case scenario in each column.

Applying the Induced l_1-Norm

Because every column of $\mathbf{S}$ contains exactly one 1 and zeros elsewhere, its column sum is strictly 1, hence $\|\mathbf{S}\|_1 = 1$.

By the multiplicative property of norms ($\|\mathbf{AB}\| \leq \|\mathbf{A}\|\|\mathbf{B}\|$), we establish:

$$\|\mathbf{S}^k\|_1 \leq \|\mathbf{S}\|_1^k = 1^k = 1 \tag{3.4}$$

This allows us to bound the error:

$$\|\mathbf{E}_N\|_1 \leq \sum_{k=N}^{\infty} \frac{\|\mathbf{S}\|_1^k}{k!} = \sum_{k=N}^{\infty} \frac{1}{k!} \tag{3.5}$$

Bounding the Error

Expanding the summation and factoring out the first neglected term, $\frac{1}{N!}$, yields:

$$\|\mathbf{E}_N\|_1 \leq \frac{1}{N!} + \frac{1}{(N+1)!} + \frac{1}{(N+2)!} + \dots \tag{3.6}$$

$$\|\mathbf{E}_N\|_1 \leq \frac{1}{N!}\left(1 + \frac{1}{N+1} + \frac{1}{(N+1)(N+2)} + \dots\right) \tag{3.7}$$

Constructing an Algebraic Bound

To construct a solvable algebraic bound, we replace the increasing denominators with the worst-case constant, N, as $\frac{1}{N+1} < \frac{1}{N}$.

This substitution results in a standard infinite geometric series:

$$\|\mathbf{E}_N\|_1 \leq \frac{1}{N!}\left(1 + \frac{1}{N} + \frac{1}{N^2} + \dots\right) = \frac{1}{N!}\sum_{m=0}^{\infty}\left(\frac{1}{N}\right)^m \tag{3.8}$$

$$\|\mathbf{E}_N\|_1 \leq \frac{1}{N!}\left(\frac{1}{1 - 1/N}\right) = \frac{1}{N!}\left(\frac{N}{N-1}\right) \tag{3.9}$$

Defining Error Tolerance

The problem requires the evaluation to be correct up to 3 decimal places.

To guarantee this precision, the absolute truncation error must be strictly less than half of the place value of the third decimal digit (10^{-3}).

Therefore, our target error tolerance (ϵ) is bounded by:

$$\epsilon \leq \frac{1}{2} \times 10^{-3} = 0.0005 \tag{3.10}$$

Enforcing Tolerance Limits

For 3 decimal places of precision, we apply error tolerance ($\epsilon \leq 0.0005$) on our derived bound:

$$\frac{1}{N!}\left(\frac{N}{N-1}\right) \leq 0.0005 \tag{3.11}$$

Checking values of the filter length N:

$$N = 5: \quad \frac{1}{120}\left(\frac{5}{4}\right) = \frac{1}{96} \approx 0.01042 \quad \text{(Not met)} \tag{3.12}$$

$$N = 6: \quad \frac{1}{720}\left(\frac{6}{5}\right) = \frac{1}{600} \approx 0.00166 \quad \text{(Not met)} \tag{3.13}$$

$$N = 7: \quad \frac{1}{5040}\left(\frac{7}{6}\right) = \frac{7}{30240} \approx 0.00023 \quad \text{(Met)} \tag{3.14}$$

Comparison with Classical Lagrange Bound

The standard method to solve this question is to use Lagrange Error Bound for N terms (highest power $n = N - 1$):

$$R_{N-1}(x) = \frac{e^c}{N!}x^N \quad \text{where } c \in (0, 1) \tag{3.15}$$

Evaluating at $x = 1$ and knowing $e < 3$, the classical upper bound is:

$$R_{N-1}(1) < \frac{3}{N!} \tag{3.16}$$

Testing our derived answer of $N = 7$:

$$R_6(1) < \frac{3}{5040} \approx 0.000595 \tag{3.17}$$

Result: The classical bound fails at our ≤ 0.0005 tolerance! Our derived bound (≈ 0.00023) is significantly tighter, proving $N = 7$ is sufficient. Thus, due to having to guess an upper limit for e, the classical method performs worse than our method.

Conclusion

Thus minimum number of required terms is 7.

The Maclaurin series for e^x evaluated at $x = 1$ is:

$$e^1 = \sum_{k=0}^{\infty} \frac{1}{k!} = 1 + 1 + \frac{1}{2!} + \frac{1}{3!} + \dots \tag{3.18}$$

If we take N terms, the sum stops at the index $k = N - 1$:

$$S_N = \sum_{k=0}^{N-1} \frac{1}{k!} \tag{3.19}$$

Our target of $N = 7$ terms expands to a 6th-degree polynomial:

$$S_7 = \underbrace{1}_{k=0} + \underbrace{1}_{k=1} + \underbrace{\frac{1}{2!}}_{k=2} + \underbrace{\frac{1}{3!}}_{k=3} + \underbrace{\frac{1}{4!}}_{k=4} + \underbrace{\frac{1}{5!}}_{k=5} + \underbrace{\frac{1}{6!}}_{k=6} = 2.7181 \tag{3.20}$$

Python Implementation I

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import factorial

e_val = np.exp(1)

# start at 3 to avoid div by zero in the N/(N-1) bound
N = np.arange(3, 11)

errs = []
bounds = []

for n in N:
    k = np.arange(n)
    approx = np.sum(1 / factorial(k))

    errs.append(np.abs(e_val - approx))
    bounds.append((1 / factorial(n)) * (n / (n - 1)))

plt.figure(figsize=(9, 5))

plt.plot(N, errs, marker='o', label='Actual Error')
```

Python Implementation II

```
plt.plot(N, bounds, marker='s', ls='--', label='Theoretical Bound')

plt.axhline(y=0.0005, color='r', ls=':', label='Tolerance (0.0005)')
plt.axvline(x=7, color='g', ls=':', alpha=0.5)

plt.annotate('N=7', xy=(7, 0.0005), xytext=(7.3, 0.005),
            arrowprops=dict(facecolor='black', shrink=0.05, width=1, headwidth
                =5))

plt.yscale('log')
plt.xlabel('Number of terms (N)')
plt.ylabel('Error')
plt.title('Truncation Error for e^x at x=1')
plt.xticks(N)
plt.grid(True, ls="--", alpha=0.4)
plt.legend()

plt.tight_layout()
path='../figs/plot.png'
plt.savefig(path, dpi=300)
print(f"Saved: {path}")
plt.close()
```

Error Plot

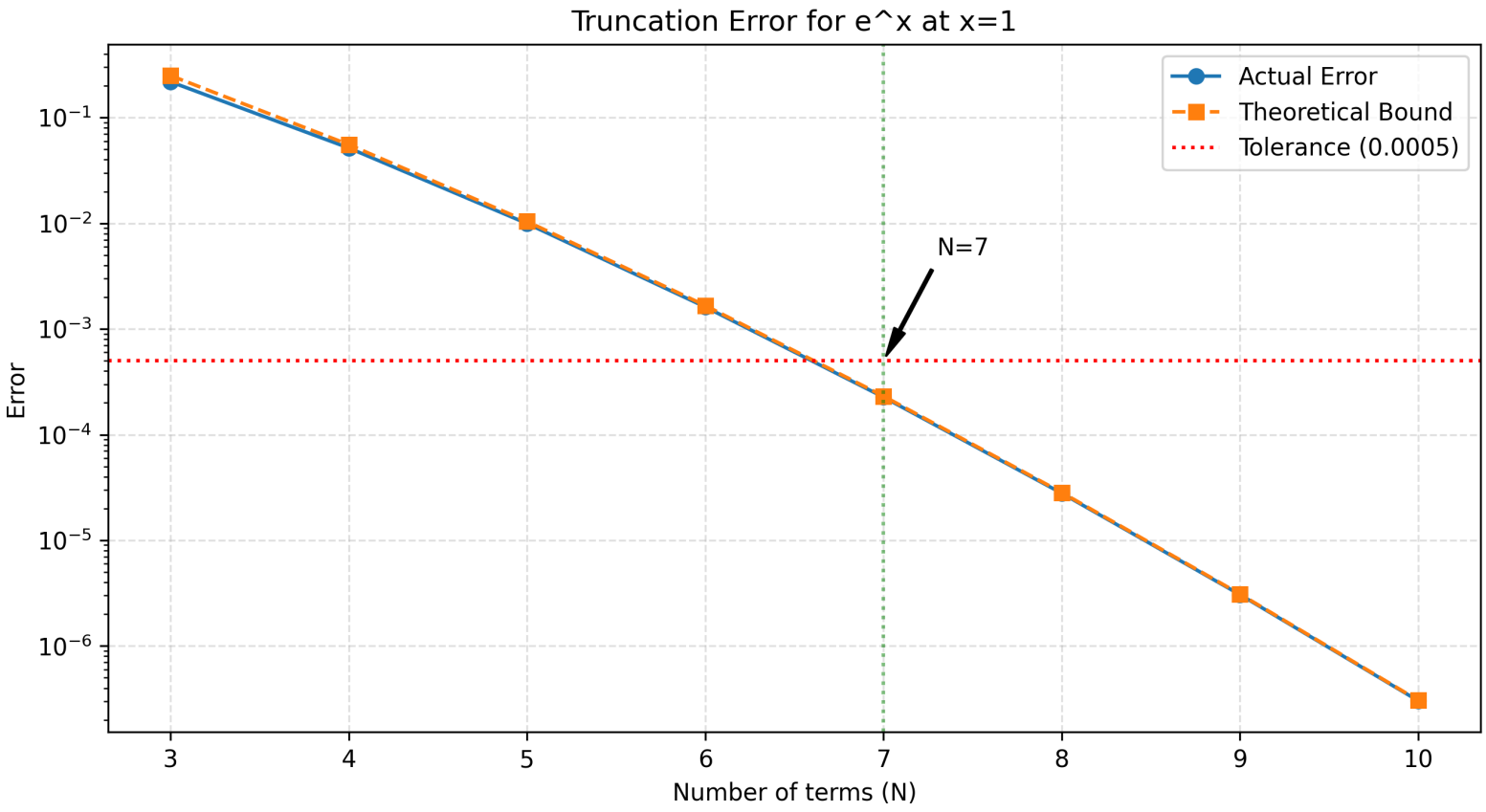

Figure: Error v/s N